CheckIn Eventos

Resumen de arquitectura y guía para la interrogación

Taller 2 · Aplicación Web Full Stack · Web Móvil 2026-2

Capa	Tecnología	Dónde está
Frontend	Angular 22 (standalone, Signals, httpResource) + Angular Material	`frontend/src/app/`
Backend	FastAPI + Pydantic v2 + Uvicorn	`backend/app/`
ORM	SQLModel (sobre SQLAlchemy 2)	`backend/app/models.py`
Base de datos	PostgreSQL 16 en Docker Compose	`docker-compose.yml`
Autenticación	bcrypt (contraseñas) + JWT HS256 (PyJWT)	`backend/app/security.py`
Pruebas	pytest: 44 pruebas backend · Vitest: 18 pruebas Angular (todas pasan)	`backend/tests/`

> **Idea central en una frase:** el usuario interactúa con componentes Angular; éstos piden datos a **servicios** que hacen solicitudes HTTP con JSON; el **interceptor** agrega el JWT; FastAPI valida el token y el JSON con **Pydantic**, opera sobre la base con el ORM **SQLModel** y responde JSON; Angular lo refleja en pantalla a través de **Signals** y **httpResource**.

1. Revisión contra la pauta

Revisé el código del repositorio (rama **main**, commit 9e7365b) requisito por requisito. Las pruebas del backend pasan (**44/44**), las del frontend también (**18/18**), el build de producción compila y la app se probó completa en el navegador contra PostgreSQL (registro, login, CRUD, búsqueda, asistencia, permisos 401/403, cascada, móvil y logout). **Resultado: la aplicación cumple todos los requisitos obligatorios.** Las observaciones de la sección 2 son detalles a corregir o a tener preparados para la defensa.

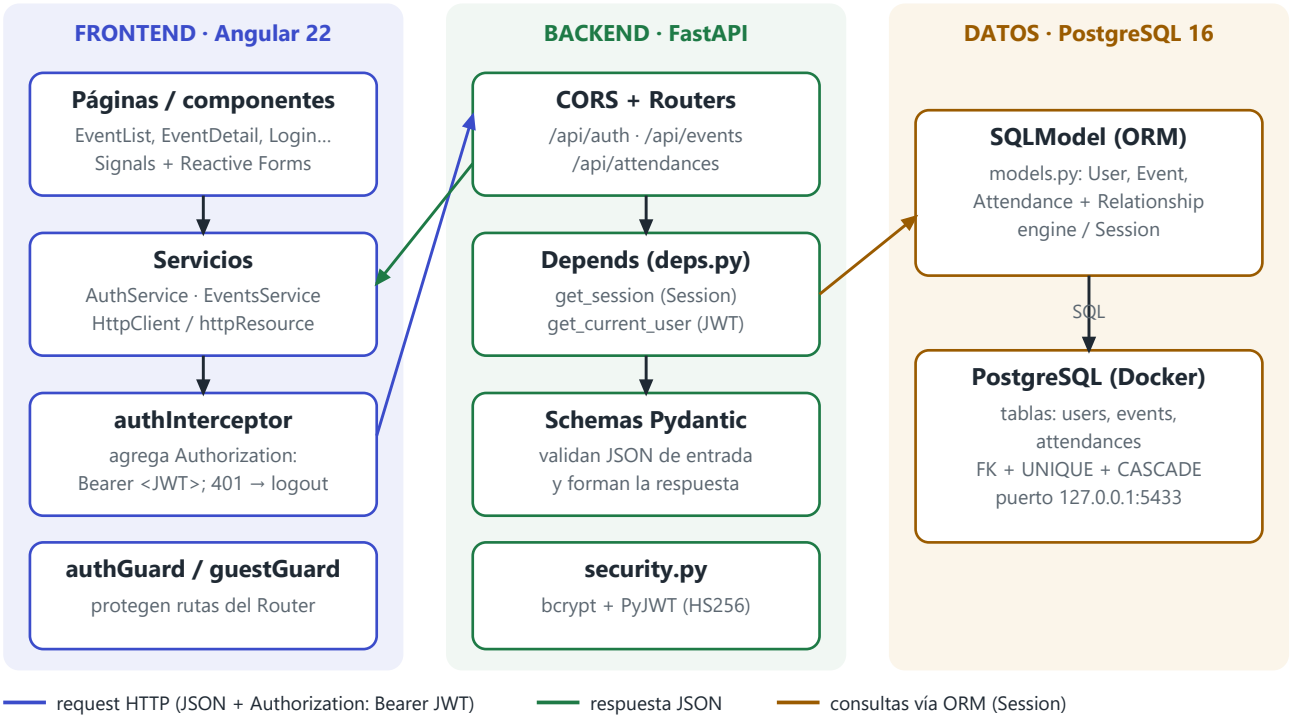
Requisito de la pauta	Implementación en el proyecto	Estado
≥ 4 vistas con Angular Router	5 rutas: /login, /registro, /eventos, /eventos/:id, /perfil (lazy `loadComponent`) en `app.routes.ts`	**Cumple**
Componentes reutilizables	EventCard, EmptyState, ConfirmDialog, EventForm (crear y editar), Nav	**Cumple**
Servicios	AuthService (`auth/auth.ts`), EventsService (`events/events.ts`)	**Cumple**
Consumo API con HTTP de Angular	`HttpClient` (POST/PUT/DELETE) y `httpResource` (GET)	**Cumple**
Signals para estado relevante	Sesión (token, usuario, autenticado), búsqueda/página, isOwner, busy, saving…	**Cumple**
resource / rxResource / httpResource	`httpResource` en listado, detalle, QR, asistentes y mis asistencias	**Cumple**
Reactive Forms + validaciones	Login, Registro y EventForm con Validators, validadores propios y mat-error	**Cumple**
Guard	`authGuard` y `guestGuard` (funcionales, `CanActivateFn`)	**Cumple**
Interceptor	`authInterceptor`: agrega Bearer y maneja 401	**Cumple**
Angular Material + responsive	Toolbar, Card, FormField/Input, Select, Button, Dialog, SnackBar, Spinner, Icon; @media 600/900px	**Cumple**
API REST FastAPI con JSON	Routers auth, events, attendances bajo /api; Swagger en /docs	**Cumple**
ORM + PostgreSQL	SQLModel + psycopg 3 contra PostgreSQL 16 (Docker)	**Cumple**
Usuarios, entidad principal, relación	users, events (principal), attendances; FK owner_id y N:M vía attendances	**Cumple**
CRUD completo desde Angular	Crear (diálogo), listar/consultar (lista + detalle), editar (diálogo), eliminar (confirmación)	**Cumple**
Registro, login y logout	/auth/register, /auth/login, /auth/me; logout borra token y signals	**Cumple**
Contraseñas seguras	bcrypt con sal por usuario; nunca se devuelve el hash (UserRead)	**Cumple**
JWT + endpoints protegidos	Dependencia `CurrentUser` en todas las rutas salvo register/login/health; 403 si no es dueño	**Cumple**
Estados de interfaz	Spinner (cargando), SnackBar (éxito/error), EmptyState (vacío/error + Reintentar), mat-error	**Cumple**
Validación y errores de API	Pydantic (422), 401, 403, 404, 409; `apiError()` los traduce a mensajes	**Cumple**
Frontend y backend separados	Carpetas frontend/ y backend/ (organización por funcionalidad, permitida)	**Cumple**
README: cómo ejecutar front, back y BD	Pasos 1–4 + pruebas; versión mínima de Node incluida	**Cumple**
Sin secretos en el repo	.env en .gitignore; .env.example sólo con valores de ejemplo; historial revisado	**Cumple**
Sin Firebase / JSON estático	Toda la persistencia pasa por PostgreSQL	**Cumple**
MatTable	No se usa (la pauta dice que no es obligatorio)	**Opcional**

2. Observaciones y puntos débiles (preparar la respuesta)

Nada de esto rompe la pauta, pero el docente puede elegir justo estos fragmentos. Los dos primeros conviene arreglarlos antes de entregar.

#	Observación	Qué hacer / qué responder
1	**Node y .env local.** Angular CLI 22 exige Node ≥ 22.22.3 o ≥ 24.15 (ya indicado en README y en `"engines"`). Además, un `.env` antiguo con puerto 5432 o sin `127.0.0.1:4200` en CORS impide arrancar.	Antes de la demo: actualizar Node y comparar el `.env` con `.env.example` (puerto 5433 y ambos orígenes CORS).
2	**El QR, las coordenadas y el radio no se validan al registrar asistencia.** `check_in` acepta lat/long pero no compara con el QR ni con la distancia al evento. La misma vista lo aclara: «El registro todavía se realiza con el botón».	Responder: el QR es un identificador único (`secrets.token_urlsafe(16)`) preparado para una siguiente iteración; hoy la validación es sesión JWT + UNIQUE(event_id, user_id). Si hay tiempo: validar el código y la distancia (Haversine ≤ radius_m) en el backend.
3	Asistentes se muestran como «Usuario #id» y el perfil como «Ver evento #id».	Detalle de UX: `AttendanceRead` no incluye nombre/título. Se podría agregar con la relación del ORM.
4	Login/Registro usan `*ngIf` y el resto usa `@if`.	Ambas sintaxis son válidas; `@if` es el control de flujo nuevo (no requiere importar CommonModule). Unificar es opcional.
5	JWT guardado en `localStorage`; no hay refresh token; el guard sólo revisa que exista un token.	Decisión de simplicidad. Si el token expiró, la primera llamada da 401 y el interceptor cierra sesión y redirige. La seguridad real está en el backend: el guard es sólo UX. Riesgo conocido: XSS (Angular sanitiza las plantillas).
6	Tablas creadas con `SQLModel.metadata.create_all` al iniciar (sin Alembic).	Suficiente para un taller; en producción se usarían migraciones para no perder datos al cambiar el modelo.
7	La carpeta `backend/venv` estuvo en el historial de Git (se quitó en ab458dd).	No contiene secretos; sólo hace pesado el repo. No requiere acción.

3. Arquitectura general



La aplicación es cliente–servidor en tres capas. Angular corre en el navegador (puerto 4200) y nunca toca la base de datos; FastAPI (puerto 8000) es el único que conoce la base y la contraseña. Como los orígenes son distintos, el backend habilita **CORS** sólo para los orígenes listados en `CORS_ORIGINS`.

Estructura de carpetas

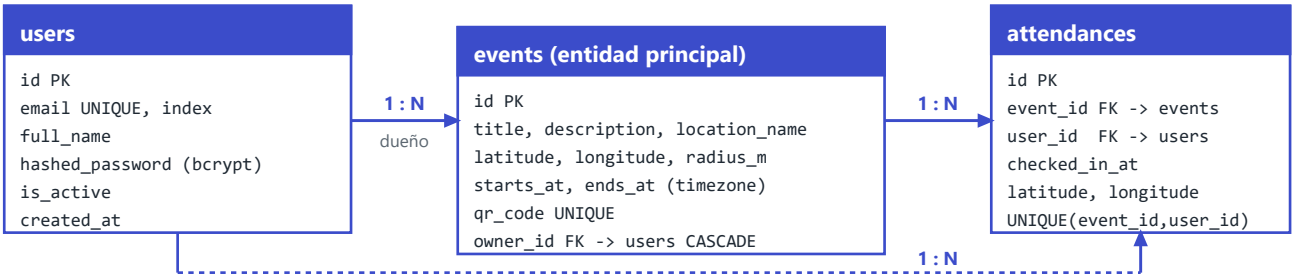
```
backend/app/                          frontend/src/app/
  main.py      app FastAPI, CORS,       app.routes.ts   rutas + guards
               routers, lifespan        app.config.ts   providers (router, http+interceptor)
  config.py    Settings desde .env      auth/           AuthService, guards, interceptor,
  database.py  engine + get_session                     login/, registro/, validadores
  models.py    tablas SQLModel          events/         EventsService, modelos TS,
  security.py  bcrypt + JWT                             event-list/, event-detail/,
  deps.py      SessionDep, CurrentUser                  event-form/, event-card/
  routers/     auth, events,            profile/        Mi perfil
               attendances              layout/nav/     barra superior
  schemas/     Pydantic (entrada/       shared/         EmptyState, ConfirmDialog,
               salida)                                  apiError()
```

Configuración y secretos

- `config.py` usa **pydantic-settings**: lee `.env` de la raíz y falla al arrancar si falta `DATABASE_URL` o si `JWT_SECRET` tiene menos de 32 caracteres (`Field(min_length=32)`).
- `.env` está en `.gitignore`; el repo sólo trae `.env.example` con valores de ejemplo.
- `docker-compose.yml` levanta PostgreSQL 16 con volumen persistente y healthcheck, expuesto sólo en 127.0.0.1.

4. Backend (FastAPI)

4.1 Modelo de datos y ORM



users 1:N events (dueño) · users 1:N attendances · events 1:N attendances → users N:M events a través de attendances.

- **SQLModel** une SQLAlchemy (tablas) y Pydantic (validación) en una clase. Cada clase con `table=True` es una tabla.
- **Relaciones:** `owner_id` es FK a users; `Relationship(back_populates=…)` permite navegar `user.events` / `event.owner`. Attendance es la tabla intermedia que relaciona usuarios y eventos (N:M).
- **Integridad en la base:** `UniqueConstraint(event_id, user_id)` impide asistencias duplicadas; `ondelete="CASCADE"` y `cascade_delete=True` borran las asistencias al borrar el evento; email y qr_code son UNIQUE.
- Fechas con `DateTime(timezone=True)` y normalizadas a UTC en el schema; Angular las muestra en la hora local.
- `database.py`: un `engine` global (pool de conexiones, `pool_pre_ping`) y `get_session()` que abre una Session por request y la cierra al terminar (`yield`).

4.2 Endpoints

Método y ruta	Protección	Qué hace / códigos
POST /api/auth/register	Pública	Crea usuario con hash bcrypt → 201 UserRead; 409 si el correo existe; 422 si el JSON es inválido
POST /api/auth/login	Pública	Verifica correo + contraseña → {access_token, token_type}; 401 genérico
GET /api/auth/me	JWT	Devuelve el usuario del token (sin hash)
GET /api/events?search&skip&limit	JWT	Lista paginada, búsqueda ILIKE en título y lugar, con attendance_count
POST /api/events	JWT	Crea evento; owner_id = usuario del token; genera qr_code → 201
GET /api/events/{id}	JWT	Detalle; 404 si no existe
PUT /api/events/{id}	JWT + dueño	Actualiza; 403 si no es el dueño
DELETE /api/events/{id}	JWT + dueño	Elimina (cascada a asistencias) → 204
GET /api/events/{id}/qr	JWT + dueño	Devuelve el código único del evento
POST /api/events/{id}/attendances	JWT	Registra asistencia del usuario → 201; 409 si ya existe
GET /api/events/{id}/attendances	JWT + dueño	Lista de asistentes del evento
GET /api/attendances/me	JWT	Asistencias del usuario actual
GET /api/health	Pública	Chequeo de salud

4.3 Autenticación y seguridad

```
# deps.py — se reutiliza en cada ruta protegida
def get_current_user(session: SessionDep, credentials = Depends(HTTPBearer(auto_error=False))) -> User:
    if credentials is None: raise 401
```

```
    user_id = decode_access_token(credentials.credentials)   # verifica firma + exp
    user = session.get(User, user_id)
    if user is None or not user.is_active: raise 401
    return user
CurrentUser = Annotated[User, Depends(get_current_user)]

@router.put("/{event_id}")
def update_event(event_id: int, data: EventUpdate, user: CurrentUser, session: SessionDep): ...
```

- **bcrypt** (`security.py`): `hashpw(password, gensalt())` — sal aleatoria por usuario y costo alto, así que un hash filtrado no se revierte con tablas precalculadas. bcrypt sólo usa 72 *bytes*; por eso schema y formulario limitan a 72 bytes (no caracteres).
- **Login sin filtrar correos:** si el correo no existe se compara igual contra `_HASH_FALSO`, así la respuesta tarda lo mismo y el mensaje es siempre «Correo o contraseña incorrectos».
- **JWT**: payload `{sub: id, iat, exp}` firmado con HS256 y `JWT_SECRET`; expira en `JWT_EXPIRE_MINUTES` (60). Es *stateless*: el servidor no guarda sesiones, sólo verifica la firma. `decode` exige `exp` y `sub`.
- **Autorización**: `require_owner()` compara `event.owner_id` con el usuario del token → 403. Ocultar botones en Angular no basta; la regla vive en el backend.

4.4 Validación y manejo de errores

- **Schemas Pydantic** separados del modelo: `UserCreate`, `LoginRequest`, `EventCreate/EventUpdate`, `EventRead`, `AttendanceCreate/Read`. La entrada se valida antes de llegar a la función; si falla, FastAPI responde **422** con el detalle por campo.
- Reglas destacadas: `EmailStr` y correo en minúsculas; `extra="forbid"` (rechaza campos desconocidos, p. ej. un owner_id inyectado); fechas con zona horaria y `ends_at > starts_at`; latitud y longitud juntas y en rango; `radius_m` entre 1 y 100000.
- **Respuesta**: `response_model` filtra los campos que salen (UserRead nunca incluye `hashed_password`).
- `HTTPException` con 401/403/404/409 y mensajes en español. Ante carreras (dos registros simultáneos), se captura `IntegrityError`, se hace rollback y se responde 409.
- **Consultas seguras y eficientes:** el ORM parametriza (sin inyección SQL); en la búsqueda se escapan `%` y `_`; los conteos de asistencia del listado se obtienen con **una** consulta `GROUP BY` (evita N+1).

5. Frontend (Angular)

5.1 Arranque, rutas y guards

```
// app.config.ts
provideRouter(routes),
provideHttpClient(withInterceptors([authInterceptor])),   // registra el interceptor

// app.routes.ts (componentes standalone cargados de forma diferida)
{ path: 'login',       canActivate: [guestGuard], loadComponent: () => import('./auth/login/login') ... },
{ path: 'registro',    canActivate: [guestGuard], ... },
{ path: 'eventos',     canActivate: [authGuard],  ... EventList },
{ path: 'eventos/:id', canActivate: [authGuard],  ... EventDetail },
{ path: 'perfil',      canActivate: [authGuard],  ... Profile },
{ path: '', redirectTo: 'eventos' }, { path: '**', redirectTo: 'eventos' }

// auth-guard.ts
export const authGuard: CanActivateFn = () =>
  inject(AuthService).autenticado() || inject(Router).createUrlTree(['/login']);
```

- **authGuard** deja pasar si hay sesión; si no, devuelve un `UrlTree` que redirige a /login. **guestGuard** hace lo inverso (un usuario logueado no ve login/registro).
- El proyecto incluye **@angular/ssr** pero en modo `RenderMode.Client`; por eso hay chequeos `typeof window`, `afterNextRender` e `isPlatformBrowser` antes de usar localStorage o generar el QR.

5.2 Interceptor

```
export const authInterceptor: HttpInterceptorFn = (request, next) => {
  const auth = inject(AuthService);
  const isApi = request.url.startsWith(environment.apiUrl + '/');
  if (isApi && auth.token())
    request = request.clone({ setHeaders: { Authorization: 'Bearer ' + auth.token() } });
  return next(request).pipe(catchError(error => {
    if (isApi && error.status === 401) { auth.logout(); router.navigate(['/login']); }
    return throwError(() => error);
  }));
};
```

- Las requests son **inmutables**: por eso se usa `clone()`.
- Sólo agrega el token a URLs de nuestra API (no lo filtra a terceros).
- Centraliza el 401: token vencido o inválido → cierra sesión y redirige, sin repetir lógica en cada componente.

5.3 Servicios

- **AuthService**: `login()` (POST, guarda token en localStorage + signal, luego `switchMap` a `/auth/me`), `registrar()`, `cargarPerfil()`, `logout()`.
- **EventsService**: lecturas como `httpResource` (`listResource, detailResource, qrResource, attendeesResource, myAttendancesResource`) y escrituras con `HttpClient` (`createEvent, updateEvent, deleteEvent, attendEvent`).
- Los componentes no arman URLs ni conocen la API: sólo llaman al servicio (requisito de la pauta y RA6).

5.4 Signals y httpResource

```
// event-list.ts
readonly searchQuery = signal('');
readonly page = signal(0);
readonly pageSize = signal(6);
readonly resource = this.service.listResource(() => ({
  search: this.searchQuery(), skip: this.page() * this.pageSize(), limit: this.pageSize() + 1,
}));
readonly events  = computed(() => this.resource.hasValue() ? this.resource.value().slice(0, this.pageSize()) : []);
readonly hasNext = computed(() => this.resource.hasValue() && this.resource.value().length > this.pageSize());

// events.ts
listResource(query) { return httpResource<EventRecord[]>(() => ({ url: this.url, params: query() })); }
```

- **Cómo funciona:** `httpResource` ejecuta la función de request dentro de un contexto reactivo. Al cambiar un signal que lee (búsqueda, página o tamaño), **vuelve a pedir los datos solo** y cancela la petición anterior. No hay `subscribe` manual ni `ngOnInit`.
- Expone signals: `value()`, `hasValue()`, `isLoading()`, `error()` y `reload()`. El template usa esos estados directamente para spinner / error / vacío / lista.
- Si la función devuelve `undefined`, no se hace request: así `qrResource` y `attendeesResource` sólo consultan si `isOwner()` es true.
- Truco de paginación: se piden `pageSize + 1` elementos; si llega uno extra, existe página siguiente.
- En el detalle, el id de la ruta se convierte a signal con `toSignal(route.paramMap…)` y alimenta `detailResource`.
- **Signals de estado**: `AuthService.token / usuario` y `autenticado = computed(...)` (los usa la Nav y los guards); `isOwner`, `registered`, `busy`, `saving`, `cargando`. Un `effect()` genera la imagen del QR (librería `qrcode`) cuando cambia el código.
- Las escrituras usan HttpClient (Observable) porque son acciones puntuales; al terminar se llama `resource.reload()` para refrescar.

5.5 Formularios reactivos

- **Login**: email (required, email) y password (required).
- **Registro**: full_name (required, 2–120, validador `fullName` que ignora espacios), email, password (8–72 + `passwordBytes` que mide bytes UTF-8 igual que el backend).
- **EventForm** (mismo componente para crear y editar; recibe el evento por `MAT_DIALOG_DATA`): título obligatorio y no vacío, largos máximos, fechas requeridas, validadores de grupo `eventDates` (término > inicio) y coordenadas pareadas, radio entero 1–100000.
- Mensajes con `mat-error`; botón deshabilitado si el form es inválido o está guardando; `markAllAsTouched()` al enviar.
- Las validaciones se repiten en el backend: en el cliente son UX, en el servidor son seguridad.

5.6 Angular Material y estados de interfaz

Estado	Cómo se muestra
Cargando	`mat-spinner` con `resource.isLoading()`; botones con texto «Guardando…/Procesando…»
Éxito	`MatSnackBar`: «Evento creado», «Evento eliminado», «Asistencia registrada», «Inicio de sesión exitoso»
Error	SnackBar + EmptyState con botón Reintentar; `apiError()` traduce detail de FastAPI (string o lista 422) y status 0 (sin conexión)
Listado vacío	`app-empty-state` «No hay eventos para mostrar» / «Aún no hay asistentes»
Form inválido	`mat-error` por campo + mensajes de grupo con role=alert
Confirmación	`MatDialog` con ConfirmDialog antes de eliminar

Responsive: grillas con `@media (max-width: 600/900px)` en Nav, listado, detalle y formulario; diálogos con `maxWidth: '95vw'`.

6. Flujos completos Angular → FastAPI → PostgreSQL

Este es el 35% de la interrogación: saber recorrer un flujo de punta a punta nombrando archivo, función y dato que viaja.

A. Registro

1	Usuario llena el form reactivo en `registro.ts`; si es válido, `authService.registrar()` hace POST /api/auth/register con JSON {email, full_name, password}.
2	FastAPI valida con `UserCreate` (422 si falla). `register()` busca el correo (409 si existe).
3	`hash_password()` genera hash bcrypt; se crea `User`, `session.add + commit` → INSERT en users.
4	Responde 201 con `UserRead` (sin hash). Angular muestra SnackBar y navega a /login.

B. Inicio de sesión y carga del perfil

1	`login.ts` → `AuthService.login()` → POST /api/auth/login {email, password}.
2	Backend: SELECT user por email; `verify_password()` con bcrypt; `create_access_token(user.id)` → JSON {access_token, token_type:'bearer'}.
3	Angular guarda el token en localStorage y en el signal `token`; `autenticado()` pasa a true y la Nav cambia sola.
4	`switchMap` → GET /api/auth/me; el interceptor agrega `Authorization: Bearer …`; el backend lo decodifica en `get_current_user` y devuelve el usuario → signal `usuario`.
5	Router navega a /eventos; `authGuard` lo permite. Al recargar la página, el constructor de AuthService relee el token y vuelve a pedir /auth/me.

C. Listar y buscar eventos (httpResource)

1	El usuario escribe en el buscador → `actualizarBusqueda()` hace `searchQuery.set()` y `page.set(0)`.
2	El cambio de signal re-evalúa la request de `listResource`: GET /api/events?search=…&skip=0&limit=7 (con JWT del interceptor).
3	FastAPI: valida query params (`Query(ge, le, max_length)`), arma `select(Event).where(title ILIKE … OR location ILIKE …)`, ordena, pagina y cuenta asistencias con GROUP BY.
4	Responde lista JSON de `EventRead`. `resource.value()` se actualiza → `computed events()` → el `@for` pinta `app-event-card`.

D. Crear / editar evento

1	Botón «Crear evento» abre `MatDialog` con `EventForm` (sin data = crear; con data = editar).
2	`guardar()`: valida, convierte fechas locales a ISO UTC (`toISOString()`) y llama `createEvent` (POST) o `updateEvent` (PUT /events/{id}).
3	Backend valida `EventCreate` (fechas, coordenadas, extra=forbid). Crear: owner_id = usuario del token y qr_code aleatorio. Editar: `find_event` (404) + `require_owner` (403) + `sqlmodel_update`.
4	COMMIT en events; responde `EventRead`. Angular: SnackBar, cierra el diálogo con el evento y el padre hace `resource.reload()`.

E. Eliminar evento

1	`eliminar()` abre `ConfirmDialog`; si confirma, `deleteEvent(id)` → DELETE /api/events/{id}.
2	Backend: 404/403 según corresponda; `session.delete(event)` + commit; la cascada elimina sus attendances. Responde 204 sin cuerpo.
3	Angular: SnackBar «Evento eliminado» y navega a /eventos.

F. Registrar asistencia

1	En el detalle, `registrarAsistencia()` → POST /api/events/{id}/attendances con {} (busy = true deshabilita el botón).
2	Backend: verifica que el evento exista, que no haya asistencia previa (409) y crea `Attendance(event_id, user_id)`; la restricción UNIQUE cubre el caso concurrente.
3	Angular recarga detalle (contador), asistentes y `myAttendance`; `registered()` pasa a true y el botón muestra «Asistencia registrada».

7. Preguntas probables y cómo justificar

¿Por qué FastAPI?

Tipado con Python, validación automática con Pydantic, inyección de dependencias (`Depends`) y documentación OpenAPI/Swagger generada sola en /docs.

¿Por qué un ORM y por qué SQLModel?

Trabajamos con clases Python en vez de SQL a mano, las consultas quedan parametrizadas (sin inyección SQL) y podemos cambiar de motor (las pruebas usan SQLite en memoria y la app PostgreSQL con el mismo código). SQLModel une el modelo de tabla con Pydantic.

¿Qué es Depends y para qué lo usan?

Inyección de dependencias: FastAPI llama a `get_session` y `get_current_user` antes de la ruta y le pasa el resultado. Una sesión de BD por request y la autenticación escrita una sola vez.

¿Por qué JWT y no sesiones en servidor?

El token es autocontenido y firmado: el backend no guarda estado, sólo verifica firma y expiración. Encaja con una API REST consumida por un cliente separado (web o móvil).

¿Qué contiene el JWT? ¿Se puede modificar?

`sub` (id), `iat`, `exp`. Se puede leer (base64) pero no modificar: si cambia el payload, la firma HS256 con el secreto ya no coincide y `decode` lo rechaza. Por eso no se pone información sensible.

¿Por qué bcrypt y no SHA-256?

bcrypt es lento a propósito y tiene sal por usuario: encarece la fuerza bruta y evita rainbow tables. SHA-256 es rápido, pensado para integridad, no para contraseñas.

Si alguien quita el guard en el navegador, ¿puede ver datos?

No. El guard es sólo navegación/UX. Sin un JWT válido el backend responde 401 en todas las rutas protegidas, y las acciones de dueño devuelven 403.

¿Para qué sirve el interceptor?

Evita repetir el header Authorization en cada servicio y centraliza la reacción al 401 (logout + redirección).

¿Qué diferencia hay entre Signals y Observables? ¿Por qué httpResource?

Un signal es un valor reactivo sincrónico que Angular rastrea para actualizar la vista; un Observable es un flujo asíncrono. httpResource conecta ambos: la request depende de signals y el resultado se expone como signals (valor, carga, error), así no hay subscribe/unsubscribe manual y se refetchea solo al cambiar los filtros.

¿Qué es computed y qué es effect?

`computed` deriva un valor de otros signals y se recalcula sólo cuando cambian (p. ej. `autenticado`, `isOwner`). `effect` ejecuta un efecto secundario cuando cambian (generar la imagen del QR).

¿Por qué Reactive Forms?

El modelo del formulario vive en TypeScript: validadores propios y de grupo (fechas, coordenadas), tipado y fácil de probar.

¿Por qué validar en ambos lados?

En Angular para dar feedback inmediato; en FastAPI porque el cliente se puede saltar (Postman, curl). El backend es la fuente de verdad.

¿Qué pasa con CORS?

El navegador bloquea peticiones entre orígenes distintos (4200 → 8000) salvo que el servidor lo permita. `CORSMiddleware` permite sólo los orígenes configurados.

¿Cómo evitan asistencias duplicadas?

Doble defensa: consulta previa en el endpoint (409 con mensaje claro) y `UniqueConstraint` en la base para el caso de dos clicks simultáneos (IntegrityError → 409).

¿Dónde están los secretos?

En `.env` (ignorado por Git). El README explica cómo crearlo desde `.env.example` y generar `JWT_SECRET` con `secrets.token_urlsafe(48)`.

¿Por qué componentes standalone y lazy loading?

No necesitan NgModules; cada ruta carga su componente con `loadComponent` sólo cuando se visita, reduciendo el bundle inicial.

> **Cómo ejecutar para la demo:** `docker compose up -d db` → en backend/: `python -m uvicorn app.main:app --reload` → en frontend/: `npm start` (con Node ≥ 24.15) → abrir http://127.0.0.1:4200 y http://127.0.0.1:8000/docs.